

Chương 2

NỀN TẢNG CÔNG NGHỆ DỮ LIỆU LỚN

Vũ Ngọc Thanh Sang

Faculty of Information Technology
Saigon University

01-2026

Mục tiêu bài học

1. Công nghệ lõi

- **Google Stack:** Bộ ba công nghệ tiên phong:
 - *MapReduce* (Tính toán).
 - *GFS* (Lưu trữ).
 - *Bigtable* (CSDL).
- **Hadoop Ecosystem:**
 - Kiến trúc Hadoop 1.x vs 2.x.
 - Vai trò điều phối của YARN.
- **Apache Spark:**
 - Cơ chế Resilient Distributed Datasets & In-memory.
 - So sánh hiệu năng vượt trội so với MapReduce.

2. Kiến trúc & Vận hành

- **Hạ tầng:**
 - SOA ↔ Cloud ↔ Big Data.
- **Luồng dữ liệu:**
 - Lựa chọn công cụ phù hợp cho từng quy trình:
- **Nguyên lý hệ phân tán:**
 - Cơ chế chịu lỗi.
 - Đảm bảo tính toàn vẹn dữ liệu.

Database, Data Warehouse, Data Mart và Data Lake

Tiêu chí	Database	Data Warehouse	Data Mart	Data Lake
Phạm vi	Ứng dụng cụ thể	Toàn tổ chức	Theo phòng ban / nghiệp vụ	Toàn tổ chức
Loại dữ liệu	Có cấu trúc	Có cấu trúc	Có cấu trúc	Có cấu trúc, bán cấu trúc, phi cấu trúc
Cấu trúc dữ liệu	Schema định nghĩa trước	Schema-on-write	Schema-on-write (kế thừa từ DW)	Schema-on-read
Mục đích sử dụng	Ứng dụng vận hành (OLTP)	BI, báo cáo, phân tích lịch sử (OLAP)	Phân tích theo chức năng nghiệp vụ	Phân tích Big Data, ML, AI

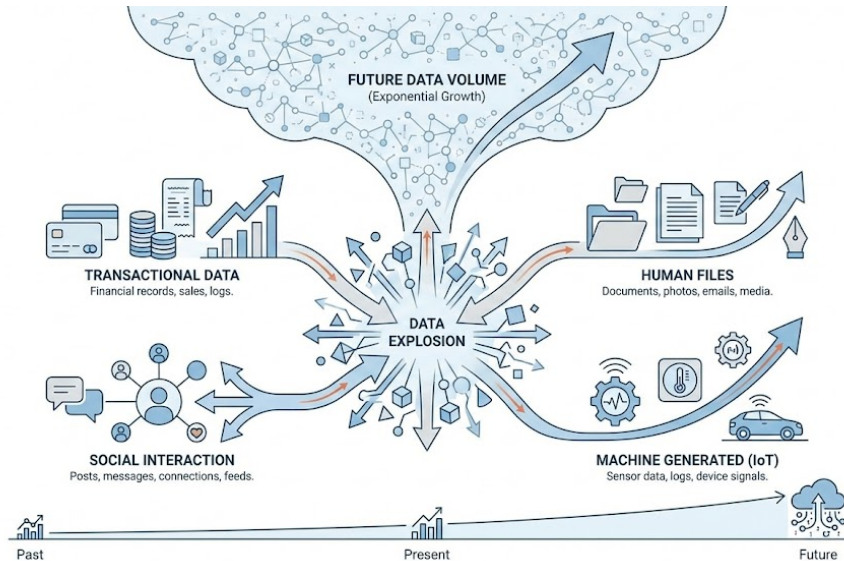
Big Data không chỉ là công cụ

- Big Data là **một giai đoạn phát triển của kiến trúc dữ liệu**, xuất phát từ nhu cầu xử lý và khai thác dữ liệu ngày càng lớn, đa dạng và phức tạp.
- Big Data mở rộng kiến trúc dữ liệu truyền thống (Database, Data Warehouse, Data Mart) sang mô hình xử lý phân tán và lưu trữ dữ liệu quy mô lớn.
- Các công nghệ như Hadoop hay Spark **chỉ là phương tiện kỹ thuật** để hiện thực hóa kiến trúc này, không phải là bản chất của Big Data.
- Trọng tâm của Big Data nằm ở **cách tổ chức, lưu trữ và khai thác dữ liệu** nhằm tạo ra giá trị cho tổ chức và hỗ trợ ra quyết định.

Bùng nổ dữ liệu & Thách thức quản lý

- **Tăng trưởng hàm mũ:** Khối lượng dữ liệu bùng nổ mạnh mẽ do sự phát triển của Internet, IoT và Điện toán đám mây.
- **Hạn chế của DBMS truyền thống:** Chỉ quản lý được một phần nhỏ dữ liệu; phần lớn dữ liệu ngày nay là *bán cấu trúc* hoặc lưu trữ cục bộ.
- **Nhu cầu cấp thiết:** Các tổ chức cần phương thức quản lý và truy cập dữ liệu *phân tán* một cách thống nhất và hiệu quả.
- **Giải pháp tất yếu:** Sự ra đời của **Hệ thống CSDL phân tán** – nay đã trở thành thành phần cốt lõi của hạ tầng công nghệ hiện đại.

Các thách thức của bùng nổ dữ liệu



Hạn chế của DBMS truyền thống

- **Scale-up đắt đỏ và giới hạn:** Việc mở rộng chủ yếu dựa vào nâng cấp phần cứng (CPU, RAM, Storage), chi phí cao và khó mở rộng vô hạn.
- **Phụ thuộc dữ liệu có cấu trúc:** DBMS truyền thống yêu cầu schema chặt chẽ, không phù hợp với dữ liệu phi cấu trúc hoặc thay đổi nhanh (log, văn bản, hình ảnh).
- **Hiệu năng giảm khi dữ liệu rất lớn:** Các phép truy vấn phức tạp trên dữ liệu khổng lồ làm giảm hiệu năng và ảnh hưởng hệ thống vận hành.
- **Không phù hợp cho phân tích quy mô lớn:** DBMS được tối ưu cho giao dịch (OLTP), không phải cho phân tích lịch sử và dữ liệu đa nguồn.

Đặc trưng 5V của Big Data

- **Volume (Khối lượng):** Dữ liệu có quy mô rất lớn, từ TB đến PB, vượt quá khả năng xử lý của hệ thống đơn lẻ.
- **Velocity (Tốc độ):** Dữ liệu được sinh ra và cần xử lý với tốc độ cao, thậm chí theo thời gian thực (streaming).
- **Variety (Đa dạng):** Dữ liệu tồn tại ở nhiều dạng khác nhau: có cấu trúc, bán cấu trúc và phi cấu trúc.
- **Veracity (Độ tin cậy):** Dữ liệu có thể nhiễu, không đầy đủ hoặc không chính xác, cần xử lý và làm sạch.
- **Value (Giá trị):** Mục tiêu cuối cùng của Big Data là trích xuất giá trị và tri thức phục vụ quyết định.

Scale Up vs Scale Out

↑ 2.1 Scale Up (Vertical Scaling)

Definition



Increase the capacity of a single node.

Characteristics



One machine becomes stronger



No change in system topology

How



+ Add more CPU



+ Add more RAM



Use faster storage (NVMe, SSD)

Example



Upgrade a database server from 32GB RAM → 128GB RAM

Limitations



Still a single point of failure



Hardware ceiling



Expensive at high end

➡ 2.2 Scale Out (Horizontal Scaling)

Definition



Increase capacity by adding more nodes.

Characteristics



Multiple machines work together



Requires distributed coordination

How



+ Add database nodes



Distribute data via sharding/partitioning



Replicate data across nodes

Example



Add 3 new database nodes and shard customer data by region



Benefits



High availability



Fault tolerance



Near-linear scalability

Vì sao cần xử lý phân tán

- **Giới hạn vật lý của máy đơn:** Một máy tính đơn lẻ không thể lưu trữ và xử lý hiệu quả dữ liệu có kích thước rất lớn.
- **Nhu cầu mở rộng linh hoạt:** Xử lý phân tán cho phép mở rộng bằng cách bổ sung nhiều máy tính giá rẻ (scale-out).
- **Tăng khả năng chịu lỗi:** Khi một nút gặp sự cố, các nút khác vẫn tiếp tục hoạt động, đảm bảo hệ thống ổn định.
- **Tăng hiệu năng xử lý:** Chia nhỏ bài toán và xử lý song song giúp rút ngắn thời gian xử lý dữ liệu lớn.

MapReduce – Ý tưởng cốt lõi

- Được Google công bố năm 2004 nhằm giải quyết bài toán xử lý dữ liệu khổng lồ từ hệ thống tìm kiếm.
- Cung cấp một mô hình lập trình đơn giản để xử lý dữ liệu lớn trên **cụm máy tính phổ thông** (commodity hardware).
- Trừu tượng hóa các vấn đề phức tạp như song song hóa, phân phối dữ liệu và chịu lỗi khỏi người lập trình.

Triết lý *Move Code to Data*

- Thay vì di chuyển khối lượng dữ liệu rất lớn qua mạng, hệ thống đưa chương trình xử lý đến nơi dữ liệu đang được lưu trữ.
- Giảm đáng kể lưu lượng truyền dữ liệu và tránh tắc nghẽn băng thông mạng.
- Tận dụng tính cục bộ dữ liệu (data locality) để nâng cao hiệu năng xử lý.

Hai pha của MapReduce

- **Map:** Xử lý dữ liệu tại từng nút, chuyển đổi dữ liệu đầu vào thành các cặp khóa–giá trị trung gian.
- **Reduce:** Tổng hợp và xử lý các dữ liệu trung gian có cùng khóa để tạo ra kết quả cuối cùng.
- Mô hình hai pha giúp đơn giản hóa quá trình xử lý song song trên dữ liệu lớn.

Giới thiệu Apache Hadoop

- Apache Hadoop là nền tảng mã nguồn mở hiện thực hóa các ý tưởng từ Google File System và MapReduce.
- Được thiết kế để lưu trữ và xử lý dữ liệu lớn trên cụm máy phân tán.
- Hai thành phần cốt lõi:
 - **HDFS**: hệ thống lưu trữ dữ liệu phân tán.
 - **MapReduce**: mô hình xử lý dữ liệu phân tán.

Đặc điểm chính của Hadoop

- **Commodity hardware:** Chạy trên các máy tính phổ thông, giúp giảm chi phí đầu tư hạ tầng.
- **Scale-out:** Dễ dàng mở rộng hệ thống bằng cách thêm nút mới khi dữ liệu tăng.
- **Fault tolerance:** Hệ thống tự động sao chép dữ liệu và phục hồi khi có lỗi phần cứng.
- **Throughput-oriented:** Tối ưu cho xử lý dữ liệu khối lượng lớn hơn là độ trễ thấp.

Hadoop Distributed File System (HDFS) – Vai trò

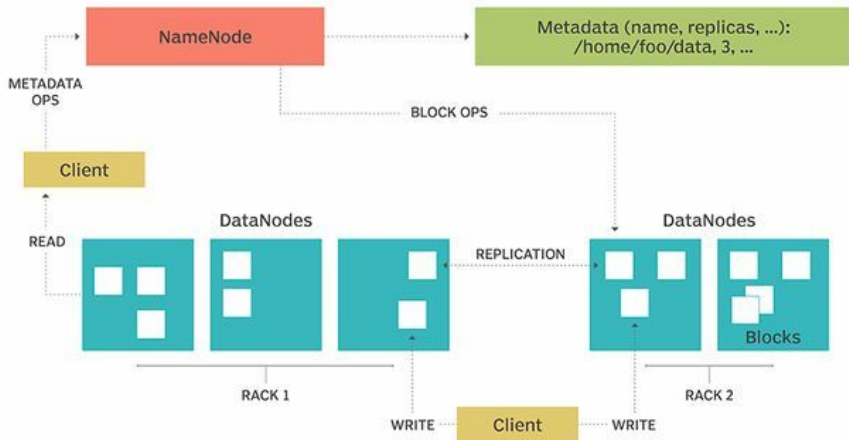
- **Lưu trữ phân tán dung lượng lớn:** Cho phép lưu trữ dữ liệu từ hàng TB đến PB trên cụm nhiều máy tính phổ thông (commodity hardware).
- **Tối ưu throughput:** Thiết kế để đọc/ghi dữ liệu khối lượng lớn theo luồng (batch processing), không tối ưu cho truy cập ngẫu nhiên.
- **Hỗ trợ xử lý phân tán:** Dữ liệu được đặt gần nơi xử lý (data locality), hỗ trợ hiệu quả cho MapReduce và Apache Spark.
- **Khả năng chịu lỗi cao:** Dữ liệu được sao chép tự động trên nhiều nút, đảm bảo hệ thống vẫn hoạt động khi phần cứng gặp sự cố.
- **Nền tảng lưu trữ của hệ sinh thái Hadoop:** Là lớp lưu trữ cốt lõi cho các công cụ phân tích như Hive, HBase, Spark và các ứng dụng Big Data khác.

HDFS – Kiến trúc

- **NameNode:** Quản lý metadata của hệ thống file như cấu trúc thư mục, vị trí các block dữ liệu.
- **DataNode:** Lưu trữ các block dữ liệu thực tế và phục vụ đọc/ghi dữ liệu.
- **Cơ chế sao chép:** Mỗi block dữ liệu được sao chép trên nhiều DataNode để đảm bảo an toàn dữ liệu.

HDFS – Kiến trúc

HDFS architecture



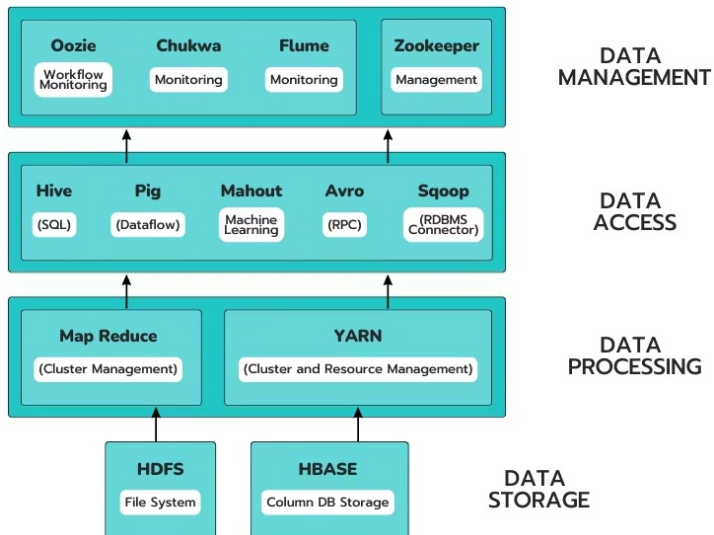
Hadoop 2 và YARN

- Hadoop 2 giới thiệu **YARN (Yet Another Resource Negotiator)** để tách quản lý tài nguyên khỏi mô hình xử lý MapReduce.
- YARN cho phép nhiều engine xử lý khác nhau (MapReduce, Spark, Tez) chạy chung trên cùng một cụm Hadoop.
- Nhờ YARN, Hadoop trở thành **nền tảng dữ liệu đa mục đích**, không chỉ giới hạn ở MapReduce.

Hệ sinh thái Hadoop

- **HDFS**: lưu trữ dữ liệu phân tán.
- **YARN**: quản lý tài nguyên và điều phối ứng dụng.
- **Hive**: truy vấn dữ liệu lớn bằng ngôn ngữ gần SQL.
- **HBase**: cơ sở dữ liệu NoSQL chạy trên HDFS.
- **ZooKeeper**: điều phối và đồng bộ các dịch vụ phân tán.

Hệ sinh thái Hadoop



Ứng dụng thực tế của Hadoop

- **Log analysis:** Phân tích log truy cập website, hệ thống, ứng dụng ở quy mô lớn.
- **Fraud detection:** Phát hiện gian lận trong tài chính, ngân hàng, thương mại điện tử.
- **Retail analytics:** Phân tích hành vi khách hàng, xu hướng mua sắm và tối ưu chuỗi cung ứng.

Vì sao Spark ra đời

- MapReduce hoạt động theo mô hình batch và ghi dữ liệu trung gian ra đĩa, gây chậm đối với các bài toán lặp.
- Nhu cầu phân tích tương tác (interactive analytics) và xử lý dữ liệu nhanh hơn ngày càng tăng.
- Sự phát triển của Machine Learning đòi hỏi khả năng xử lý dữ liệu lặp lại nhiều lần trên cùng tập dữ liệu.

Apache Spark là gì

- Apache Spark là một **engine xử lý dữ liệu phân tán** với khả năng xử lý dữ liệu chủ yếu trong bộ nhớ (in-memory).
- Hỗ trợ xử lý dữ liệu lớn nhanh hơn nhiều so với MapReduce truyền thống.
- Có thể chạy trên Hadoop (qua YARN, HDFS) hoặc hoạt động độc lập với hệ thống lưu trữ riêng.

Ưu điểm của Spark

- **Hiệu năng cao:** Tận dụng bộ nhớ để giảm số lần truy cập đĩa, giúp xử lý nhanh hơn.
- **Đa ngôn ngữ:** Hỗ trợ Scala, Python, Java và R, phù hợp với nhiều đối tượng người dùng.
- **Đa mô hình xử lý:** Hỗ trợ batch processing, streaming, machine learning và graph processing trong cùng một nền tảng.

RDD – Khái niệm

- RDD (Resilient Distributed Dataset) là tập dữ liệu phân tán, được chia thành nhiều partition trên các nút khác nhau.
- RDD là **bất biến (immutable)**, mọi phép biến đổi đều tạo ra RDD mới.
- Thiết kế này giúp đơn giản hóa xử lý song song và đảm bảo an toàn dữ liệu.

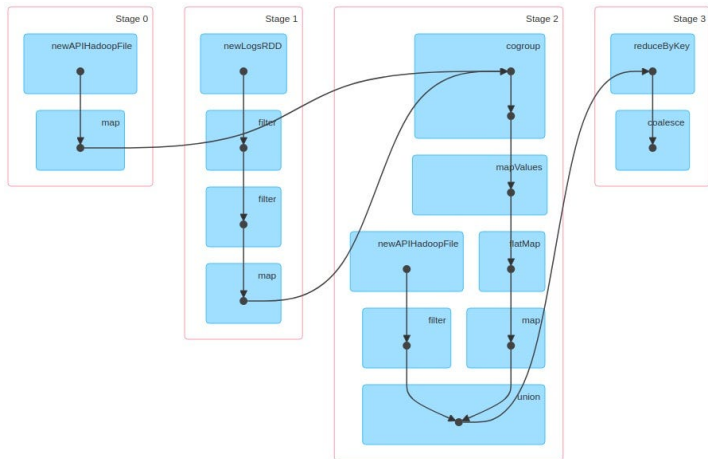
Lineage & Fault Tolerance

- Spark không cần sao chép dữ liệu trung gian như Hadoop MapReduce, giúp giảm I/O đĩa và tăng hiệu năng.
- Mỗi RDD lưu lại **lineage** – chuỗi các phép biến đổi đã tạo ra nó từ dữ liệu gốc.
- **Lineage (phả hệ dữ liệu):** Là đồ thị mô tả quan hệ phụ thuộc giữa các RDD, cho biết dữ liệu được tạo ra thông qua những bước xử lý nào.
- **Fault Tolerance (khả năng chịu lỗi):** Là khả năng hệ thống tiếp tục hoạt động và khôi phục dữ liệu khi một nút hoặc phân vùng gặp sự cố.
- Khi xảy ra lỗi, Spark tự động tái tạo lại phân vùng dữ liệu bị mất bằng cách thực thi lại các phép biến đổi dựa trên lineage, thay vì phục hồi từ bản sao lưu.

DAG Execution Model

- Spark xây dựng đồ thị có hướng không chu trình (DAG) để biểu diễn toàn bộ pipeline xử lý.
- DAG cho phép hệ thống tối ưu hóa thứ tự và cách thức thực thi các phép biến đổi.
- Nhờ đó, Spark giảm tối đa việc ghi dữ liệu trung gian ra đĩa.

Ví dụ DAG



Spark vs MapReduce

- **MapReduce**: Xử lý batch, ghi dữ liệu trung gian ra đĩa, phù hợp với các tác vụ đơn giản.
- **Spark**: Xử lý in-memory, hỗ trợ phân tích tương tác và các bài toán phức tạp.
- Spark đặc biệt hiệu quả cho các tác vụ lặp và phân tích nâng cao.

Vai trò Spark trong Big Data

- Spark là **engine phân tích trung tâm** trong nhiều hệ thống Big Data hiện đại.
- Đóng vai trò cầu nối giữa lưu trữ dữ liệu lớn (Data Lake) và phân tích nâng cao.
- Là nền tảng quan trọng cho Machine Learning, Data Science và AI.

Data Architecture là gì

- Khung tổng thể mô tả cách dữ liệu được **thu thập, lưu trữ, xử lý và khai thác** trong một hệ thống.
- Xác định **luồng dữ liệu** từ hệ thống vận hành đến phân tích và ra quyết định.
- Kết nối giữa **công nghệ dữ liệu** và **mục tiêu nghiệp vụ**.
- Đảm bảo dữ liệu có thể mở rộng, an toàn và dễ khai thác.

Database – Data Warehouse

- **Database (OLTP)** phục vụ vận hành hằng ngày của hệ thống.
- **Data Warehouse (OLAP)** phục vụ phân tích, báo cáo và ra quyết định.
- Database tối ưu cho **ghi/đọc nhanh**, Data Warehouse tối ưu cho **truy vấn phân tích**.
- Data Warehouse tích hợp dữ liệu từ nhiều Database khác nhau.

Data Mart

- Data Mart là một **tập con có trọng tâm** của Data Warehouse.
- Phục vụ phân tích theo **phòng ban hoặc nghiệp vụ cụ thể**.
- Giúp đơn giản hóa truy vấn và cải thiện hiệu năng phân tích.
- Không tồn tại độc lập, phụ thuộc vào Data Warehouse.

Data Lake

- Lưu trữ dữ liệu ở dạng **thô, chưa xử lý**.
- Hỗ trợ mọi loại dữ liệu: có cấu trúc, bán cấu trúc và phi cấu trúc.
- Áp dụng mô hình **schema-on-read**.
- Nền tảng cho Big Data analytics, Machine Learning và AI.

Thứ tự phát triển của kiến trúc dữ liệu

- Database xuất hiện đầu tiên để phục vụ vận hành hệ thống.
- Data Warehouse ra đời khi cần phân tích và ra quyết định.
- Data Mart hỗ trợ phân tích theo từng bộ phận.
- Data Lake xuất hiện khi dữ liệu trở nên rất lớn và đa dạng.

Ảnh xạ công cụ trong kiến trúc dữ liệu

- Database: MySQL, PostgreSQL, Oracle.
- Data Warehouse: Hive, Redshift, BigQuery.
- Data Mart: tập con của Data Warehouse.
- Data Lake: HDFS, Amazon S3.
- Xử lý dữ liệu: Apache Spark.

Big Data trong doanh nghiệp

- Hỗ trợ ra quyết định dựa trên dữ liệu thay vì cảm tính.
- Khai thác dữ liệu từ nhiều nguồn khác nhau trong tổ chức.
- Tăng khả năng dự báo và tối ưu hoạt động.

Big Data và AI

- Dữ liệu là nền tảng để huấn luyện các mô hình AI.
- Big Data cung cấp dữ liệu lớn và đa dạng cho Machine Learning.
- Chất lượng dữ liệu ảnh hưởng trực tiếp đến chất lượng mô hình AI.

Pipeline Big Data

- Pipeline Big Data mô tả toàn bộ vòng đời của dữ liệu trong hệ thống.
- **Acquisition**: thu thập dữ liệu từ nhiều nguồn khác nhau.
- **Storage**: lưu trữ dữ liệu với khả năng mở rộng lớn.
- **Processing**: xử lý và biến đổi dữ liệu.
- **Analytics**: phân tích và khai thác giá trị từ dữ liệu.

Thu thập dữ liệu

- Thu thập dữ liệu từ các hệ thống vận hành, ứng dụng và thiết bị.
- **Kafka**: nền tảng thu thập và truyền dữ liệu theo luồng (streaming).
- **Flume**: thu thập dữ liệu log vào hệ thống Hadoop.
- **Sqoop**: nhập dữ liệu từ cơ sở dữ liệu quan hệ vào Hadoop.

Lưu trữ dữ liệu

- Lưu trữ dữ liệu với dung lượng lớn và khả năng mở rộng.
- **HDFS**: lưu trữ file phân tán, tối ưu cho dữ liệu lớn.
- **HBase**: cơ sở dữ liệu NoSQL cho truy cập ngẫu nhiên trên HDFS.
- Đảm bảo tính sẵn sàng và an toàn dữ liệu.

Xử lý dữ liệu

- Thực hiện biến đổi, tổng hợp và làm sạch dữ liệu.
- **Spark**: engine xử lý phân tán hiệu năng cao.
- **Hive**: xử lý dữ liệu lớn bằng ngôn ngữ truy vấn gần SQL.
- Hỗ trợ cả batch processing và phân tích nâng cao.

Truy vấn và phân tích

- Cho phép người dùng khai thác và phân tích dữ liệu đã xử lý.
- **Hive**: truy vấn dữ liệu lớn theo mô hình OLAP.
- **Impala**: truy vấn dữ liệu lớn với độ trễ thấp.
- Phục vụ báo cáo, BI và phân tích nghiệp vụ.

Điều phối

- Điều phối và đồng bộ các thành phần trong hệ thống phân tán.
- **ZooKeeper**: cung cấp dịch vụ điều phối, cấu hình và đồng bộ.
- Hỗ trợ phát hiện lỗi và duy trì trạng thái hệ thống.

Tổng kết chương

- Hiểu được bản chất và vai trò của Big Data trong hệ thống thông tin hiện đại.
- Nắm được các ý tưởng nền tảng của xử lý dữ liệu lớn: phân tán, song song và chịu lỗi.
- Đặt đúng vai trò của Hadoop (lưu trữ, nền tảng) và Spark (xử lý, phân tích) trong hệ sinh thái Big Data.
- Hiểu mối liên hệ giữa Big Data, kiến trúc dữ liệu và Data Science.
- Sẵn sàng tiếp cận các nội dung **phân tích dữ liệu lớn và ứng dụng nâng cao**.

Thank you for your attention